

MarketAI

L'IA Consultant Marketing

Guide de Projet Complet — Budget 0\$

RAG

AI Agent

Prompt Engineering

Fine-Tuning

Startup-Ready

Projet de fin de formation en IA Générative

Construit pour devenir une startup

1. Vision du Projet

1.1 Concept

MarketAI n'est pas un chatbot. C'est un système d'intelligence artificielle multi-agent dont l'unique objectif est de générer plus de revenus pour ses clients. Là où ChatGPT répond à des questions, MarketAI exécute des missions marketing complètes.

Sa cible : les PME, artisans, artistes, créateurs de contenu et petits commerçants qui n'ont pas les moyens d'un cabinet marketing (3 000 à 10 000€/mois) mais qui ont besoin de ces services.

Proposition de valeur unique

Un consultant marketing disponible 24h/24, au prix d'un abonnement SaaS, qui connaît les meilleures pratiques mondiales, adapte ses conseils à chaque secteur et livrable des assets prêts à l'emploi.

1.2 Ce que MarketAI peut faire

Livrable	Description
Stratégie marketing complète	Positionnement, cible, canaux, budget, KPIs
Page vitrine HTML/CSS	Générée et téléchargeable en direct
Calendrier éditorial 30 jours	Posts réseaux sociaux prêts à publier
Copywriting publicitaire	Facebook Ads, TikTok, YouTube, Google Ads
Séquence emails de vente	3 à 7 mails avec taux d'ouverture optimisé
Analyse concurrentielle	Avec web scraping en temps réel
Plan de pricing	Positionnement tarifaire et psychologie des prix
Rapport de performance	Analyse et recommandations d'amélioration

1.3 Pourquoi c'est une startup viable

- Marché adressable : 3,5 millions de TPE/PME en France seule
- Prix visé : 49 à 199€/mois — entre 10x moins cher qu'un consultant
- Coût marginal proche de zéro une fois le système construit
- Différenciation forte : totalement orienté résultats financiers, pas généraliste
- Timing parfait : les APIs LLM sont matures, la demande est là

2. Stack Technique — Budget 0\$

Toutes les technologies listées ci-dessous sont gratuites, open-source ou disposent d'un tier gratuit suffisant pour construire et tester le MVP.

2.1 LLM (Cerveau principal)

Outil	Pourquoi ce choix
Groq API (free tier)	Llama 3.1 70B ultra-rapide, 30 req/min gratuit — idéal pour le développement
Google Gemini (free tier)	1 million de tokens gratuits/mois, très capable
Ollama (local)	Faire tourner Llama 3, Mistral, Phi-3 en local — 0\$ absolu
Hugging Face Inference API	Accès gratuit à des centaines de modèles
Together AI	Crédit de démarrage gratuit, modèles open-source rapides

2.2 RAG — Retrieval Augmented Generation

Composant	Outil gratuit recommandé
Base vectorielle	ChromaDB (local, open-source) ou Qdrant (cloud free tier)
Embeddings	sentence-transformers/all-MiniLM-L6-v2 (Hugging Face, gratuit)
Framework RAG	LlamaIndex ou LangChain (open-source)
Chunking & parsing	LangChain Document Loaders (PDF, web, CSV, JSON)
Reranking	cross-encoder/ms-marco-MiniLM (Hugging Face, gratuit)

2.3 Agent & Orchestration

Composant	Outil gratuit recommandé
Orchestrateur	LangGraph (LangChain) — graphe d'agents le plus robuste
Alternative	CrewAI — plus simple, idéal pour débiter
Outils web	DuckDuckGo Search API (gratuit) + BeautifulSoup pour le scraping
Mémoire court terme	LangChain ConversationBufferMemory
Mémoire long terme	ChromaDB + résumés automatiques

2.4 Fine-Tuning

Composant	Outil gratuit recommandé
Environnement	Google Colab (GPU T4 gratuit, 12h/session)
Framework	Unsloth + TRL — fine-tuning ultra-optimisé, 2x moins de VRAM
Modèle de base	Mistral 7B ou Llama 3.1 8B (Hugging Face)
Méthode	LoRA / QLoRA — adaptatif, léger, efficace
Dataset	Alpaca format JSON, construit manuellement (voir section 4)
Hébergement modèle	Hugging Face Hub (gratuit, dépôt privé possible)

2.5 Backend & Frontend

Composant	Outil gratuit recommandé
Backend API	FastAPI (Python) — rapide, async, documentation auto
Frontend	Next.js 14 (React) — ou Streamlit pour un MVP rapide
Base de données	PostgreSQL via Supabase (free tier généreux)
Cache & sessions	Redis via Upstash (free tier 10k req/jour)
Authentification	Supabase Auth (gratuit) ou Clerk (free tier)
Hébergement backend	Railway (free tier) ou Render (free tier)
Hébergement frontend	Vercel (gratuit pour projets personnels)
Génération PDF	WeasyPrint ou ReportLab (Python, open-source)

2.6 Outils de développement

Besoin	Outil gratuit
IDE	VS Code avec extensions Python + GitHub Copilot (gratuit étudiants)
Versioning	GitHub (gratuit, dépôt public ou privé)
Tests API	Postman ou Bruno (open-source)
Monitoring	LangSmith (free tier LangChain) pour tracer les agents
Variables d'env	python-dotenv — ne jamais commit les clés API

3. Ressources & Données pour le RAG

La qualité du RAG détermine 80% de la qualité des réponses. Voici exactement quoi collecter et comment l'obtenir gratuitement.

3.1 Types de données à ingérer

Données priorité HAUTE (à collecter en premier)

- Études de cas marketing réels : campagnes qui ont multiplié les ventes
- Playbooks de croissance : stratégies documentées de startups à succès
- Guides de copywriting : techniques de persuasion, formules AIDA, PAS, BAB
- Benchmarks sectoriels : taux de conversion moyens par industrie
- Exemples de pages de vente haute performance (avec métriques)

3.2 Sources gratuites par catégorie

Marketing stratégique

- HubSpot Blog (blog.hubspot.com) — des milliers d'articles, exportables en PDF
- Neil Patel Blog (neilpatel.com) — SEO, content marketing, growth hacking
- MarketingProfs.com — études et guides professionnels
- Think with Google (thinkwithgoogle.com) — études de cas avec données réelles
- Content Marketing Institute — recherches annuelles gratuites

Copywriting & publicité

- Swipe File (swipefile.com) — bibliothèque d'annonces qui convertissent
- Swiped.co — exemples de copy classiques et modernes
- AdEspresso blog — Facebook & Instagram Ads best practices
- Libro de Copywriting de Josef Ajram (traduit, libre accès)
- The Boron Letters de Gary Halbert (domaine public)

Social Media & contenu

- Sprout Social Insights (sproutsocial.com/insights) — statistiques plateformes
- Later Blog (later.com/blog) — Instagram, TikTok, Pinterest
- Buffer Resources (buffer.com/resources) — guides complets
- Hootsuite Blog — études et benchmarks annuels

E-commerce & conversion

- Shopify Blog (shopify.com/blog) — guides marchands, études de cas
- Baymard Institute (baymard.com) — recherches UX e-commerce gratuites
- ConversionXL / CXL Institute — articles de recherche en CRO
- Unbounce Blog — landing pages et tests A/B

Données structurées & datasets

- Kaggle (kaggle.com) — datasets marketing, sentiment analysis, ad performance
- Hugging Face Datasets — datasets NLP marketing déjà préparés
- Statista (articles gratuits) — statistiques marché
- Google Trends (trends.google.com) — données de tendances via API

Script Python pour collecter les données web (exemple)

Utiliser newspaper3k ou trafilatura pour extraire le texte propre des articles :

```
pip install trafilatura newspaper3k
import trafilatura
downloaded =
trafilatura.fetch_url('https://blog.hubspot.com/marketing/...')
text = trafilatura.extract(downloaded)
```

3.3 Structure recommandée de la base de données RAG

Organise tes documents en catégories pour un retrieval précis :

- marketing_strategy/ — stratégies générales et frameworks
- copywriting/ — techniques de rédaction persuasive
- social_media/ — guides par plateforme (Instagram, TikTok, LinkedIn...)
- ecommerce/ — conversion, panier, fidélisation
- local_business/ — marketing pour commerces de proximité
- case_studies/ — études de cas avec résultats chiffrés
- templates/ — modèles d'emails, posts, scripts vidéo
- market_data/ — données de marché, benchmarks sectoriels

Objectif minimal pour le MVP

- 300 à 500 documents textuels de qualité
- Chaque document entre 500 et 2000 mots
- Au moins 50 études de cas avec des métriques réelles
- 20 templates de copywriting fonctionnels

4. Données pour le Fine-Tuning

Le fine-tuning transforme un LLM généraliste en consultant marketing senior. L'objectif est qu'il parle naturellement le langage du marketing, adopte le bon ton et donne des conseils actionnables.

4.1 Format du dataset

Utilise le format Alpaca (instruction + input + output) en JSON :

Exemple de paire d'entraînement

```
{ "instruction": "Crée un slogan percutant pour une boulangerie artisanale à Lyon",  
  "input": "Nom : La Mie Dorée. Valeurs : tradition, qualité, local.",  
  "output": "Voici 3 slogans avec leur angle stratégique :\n1. 'Le goût du vrai, péri depuis 1987' (nostalgie + authenticité)..." }
```

4.2 Types de paires à créer

Catégorie	Nombre de paires cibles
Création de slogans et taglines	150 paires
Rédaction d'emails de vente	200 paires
Scripts de publicité sociale	200 paires
Stratégies marketing sectorielles	300 paires
Analyse de concurrence	100 paires
Conseils de pricing	100 paires
Calendriers de contenu	150 paires
Copywriting de landing pages	150 paires
Réponses à des problèmes marketing	200 paires
TOTAL minimum recommandé	1 500 paires

4.3 Sources pour construire le dataset

- Générer avec GPT-4 puis vérifier manuellement (technique seed-GPT)
- Transformer des études de cas en paires Q/R
- Utiliser des forums marketing (Reddit r/marketing, r/entrepreneur)
- S'inspirer de vraies conversations avec des entrepreneurs
- Extraire des Q/R de livres marketing classiques (libres de droits)

Commande de fine-tuning avec Unsloth (Google Colab)

```
from unsloth import FastLanguageModel
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/mistral-7b-instruct-v0.3-bnb-4bit",
    max_seq_length = 2048, load_in_4bit = True)
```

5. Roadmap de Développement

La roadmap est divisée en 6 phases sur 10 à 14 semaines. Chaque phase a des objectifs clairs et des livrables vérifiables. Ne pas passer à la phase suivante sans avoir validé les tests de la phase courante.

Phase 0	Préparation & environnement	Sem. 1
---------	-----------------------------	--------

Objectif

Mettre en place un environnement de développement propre et structuré avant d'écrire la moindre ligne de code métier.

Tâches

1. Créer le dépôt GitHub avec la structure de dossiers complète
2. Configurer l'environnement Python (pyenv + virtualenv)
3. Ouvrir les comptes gratuits : Groq, Google AI Studio, Hugging Face, Supabase, Vercel
4. Installer VS Code avec les extensions Python, Pylance, GitLens
5. Configurer le fichier .env et le .gitignore
6. Initialiser FastAPI avec un endpoint /health fonctionnel
7. Déployer ce skeleton sur Railway ou Render

Structure de dossiers recommandée

Arborescence du projet	
marketai/	
├─ backend/	# FastAPI
│ ├─ agents/	# LangGraph agents
│ ├─ rag/	# RAG pipeline
│ ├─ prompts/	# Templates de prompts
│ └─ tools/	# Web search, HTML gen...
└─ api/	# Routes FastAPI
├─ frontend/	# Next.js
├─ data/	# Données RAG brutes
│ ├─ raw/	
│ └─ processed/	
├─ finetuning/	# Notebooks Colab + dataset
├─ scripts/	# Collecte données, ingestion
└─ docs/	# Documentation

Critère de validation

8. Dépôt GitHub structuré et pushé
9. Endpoint /health répond 200 en production

Phase 1

Pipeline RAG

Sem. 2-3

Objectif

Construire le moteur de connaissance marketing — la base vectorielle qui permettra au système de citer des exemples réels et pertinents.

Tâches

10. Collecter 100 premiers documents (articles HubSpot, Neil Patel, études de cas)
11. Écrire le script de scraping et nettoyage (trafilatura + nettoyage regex)
12. Installer ChromaDB et tester la création d'une collection
13. Implémenter le chunking intelligent (500 tokens, overlap 50 tokens)
14. Générer les embeddings avec sentence-transformers
15. Implémenter la fonction de retrieval avec filtrage par catégorie
16. Ajouter le reranking pour améliorer la pertinence
17. Tester le RAG sur 20 questions marketing et évaluer la pertinence
18. Documenter le pipeline dans un Jupyter Notebook

Critère de validation

- Le système retrouve les 3 documents les plus pertinents en moins de 500ms
- Précision évaluée à plus de 80% sur le jeu de test

Phase 2

Prompt Engineering

Sem. 3-4

Objectif

Construire la bibliothèque de prompts optimisés qui définissent le comportement expert de MarketAI.

Templates de prompts à créer

- SYSTEM_PROMPT : persona du consultant marketing senior
- STRATEGY_TEMPLATE : génération de stratégie complète (Chain of Thought)
- COPYWRITING_TEMPLATE : avec exemples few-shot de copy qui convertissent
- SOCIAL_MEDIA_TEMPLATE : avec les spécificités de chaque plateforme
- EMAIL_TEMPLATE : séquences de vente avec hooks et CTAs
- PAGE_VITRINE_TEMPLATE : brief → structure HTML complète

- COMPETITOR_ANALYSIS_TEMPLATE : analyse structurée
- PRICING_TEMPLATE : psychologie des prix et positionnement

Techniques à appliquer

- Few-shot prompting : 3 à 5 exemples de haute qualité dans chaque template
- Chain of Thought : forcer le raisonnement étape par étape
- Role prompting : 'Tu es un consultant marketing avec 15 ans d'expérience...'
- Output formatting : contraindre le format (JSON, Markdown, HTML)
- Negative prompting : ce qu'il ne faut jamais faire/dire

Critère de validation

- Chaque template testé sur 10 inputs différents
- Score de qualité humain supérieur à 8/10 en moyenne

Phase 3	AI Agent (Orchestrateur)	Sem. 5-6
---------	--------------------------	----------

Objectif

Construire le cerveau du système — l'agent qui décompose une demande complexe en sous-tâches et les exécute dans l'ordre optimal.

Outils de l'agent à implémenter

Outil	Fonction
web_search	Recherche DuckDuckGo pour données en temps réel
rag_retrieve	Interroger la base de connaissance interne
generate_html	Créer une page vitrine complète
analyze_competitors	Scraper et analyser les concurrents
generate_calendar	Créer un calendrier éditorial structuré
calculate_roi	Estimer le retour sur investissement d'une campagne
save_to_memory	Sauvegarder le profil et l'historique client

Graphe LangGraph à implémenter

- Node 1 — Intent Classifier : détecter le type de demande
- Node 2 — Context Retriever : récupérer le contexte RAG + mémoire client
- Node 3 — Task Planner : décomposer en sous-tâches
- Node 4 — Executor : exécuter chaque sous-tâche avec le bon outil
- Node 5 — Quality Checker : vérifier et améliorer le résultat
- Node 6 — Formatter : formater selon le type de livrable

Critère de validation

- L'agent traite une demande complexe de bout en bout sans intervention humaine
- Temps de réponse inférieur à 30 secondes pour une stratégie complète

Phase 4

Fine-Tuning du modèle

Sem. 7-8

Objectif

Entraîner un modèle spécialisé qui parle naturellement le langage du marketing et donne des conseils actionnables comme un consultant senior.

Étapes

19. Constituer le dataset de 1 500+ paires (voir section 4)
20. Valider la qualité : vérifier la diversité, la cohérence, le niveau
21. Ouvrir Google Colab Pro (ou utiliser les 90 GPU-heures/mois gratuits)
22. Installer Unsloth + TRL dans le notebook
23. Lancer le fine-tuning avec QLoRA (4-bit quantization)
24. Évaluer le modèle fine-tuné vs le modèle de base sur 50 questions
25. Pousser le modèle sur Hugging Face Hub
26. Intégrer le modèle fine-tuné dans le pipeline principal

Métriques d'évaluation

- Préférence humaine : score aveugle entre fine-tuné et base (cible 70%+ de préférence)
- Pertinence marketing : expert évalue 50 réponses (cible 8+/10)
- Hallucination rate : vérification factuelle (cible moins de 5%)

Critère de validation

- Modèle fine-tuné préféré dans 70%+ des cas en test aveugle
- Modèle disponible sur Hugging Face Hub

Phase 5

Interface & Livrables avancés

Sem. 9-10

Objectif

Construire l'interface utilisateur et les fonctionnalités avancées qui font de MarketAI un vrai produit.

Frontend (Next.js)

- Interface de chat avec streaming de la réponse token par token

- Dashboard client avec historique des missions
- Téléchargement des livrables (HTML, PDF, DOCX)
- Profil entreprise : secteur, taille, concurrents, tonalité souhaitée
- Bibliothèque des pages vitrine générées

Fonctionnalités backend à finaliser

- Génération de page vitrine HTML : de brief → HTML responsive complet
- Export PDF des stratégies (WeasyPrint)
- Mémoire persistante : profil client sauvegardé entre les sessions
- Webhook pour intégrations futures (Zapier, Make)

Critère de validation

- Un utilisateur non-technique peut utiliser MarketAI sans aide
- Génération d'une page vitrine complète en moins de 60 secondes

Phase 6	Startup-Ready & lancement	Sem. 11-14
---------	---------------------------	------------

Objectif

Transformer le projet technique en produit commercial prêt à être lancé.

Tâches

27. Landing page marketing de MarketAI (Next.js + Tailwind)
28. Système de paiement Stripe (free jusqu'au premier paiement)
29. Onboarding guidé pour les nouveaux utilisateurs
30. Limite de tokens par plan (Free / Pro / Enterprise)
31. Dashboard analytics : missions réalisées, livrables générés, ROI
32. Documentation API pour partenariats futurs
33. 10 beta-testeurs recrutés (PME locales, artistes, créateurs)
34. Pitch deck investisseur (8 slides max)
35. Publier sur Product Hunt

Critère de validation

- 10 beta-utilisateurs actifs avec feedback positif documenté
- Coût mensuel infrastructure inférieur à 50€

6. Checklist de Progression

Utilise cette checklist pour suivre ta progression. Coche chaque item avant de passer à la phase suivante.

Phase 0 — Environnement

- ✓ Dépôt GitHub créé avec la structure complète
- ✓ Comptes créés : Groq, HuggingFace, Supabase, Vercel, Railway
- ✓ FastAPI endpoint /health en production

Phase 1 — RAG

- ✓ 100+ documents collectés et nettoyés
- ✓ ChromaDB opérationnel avec embeddings
- ✓ Retrieval >80% de précision sur jeu de test

Phase 2 — Prompt Engineering

- ✓ 8+ templates de prompts créés et testés
- ✓ Score humain moyen >8/10 sur chaque template

Phase 3 — Agent

- ✓ LangGraph avec 6+ outils fonctionnels
- ✓ Test end-to-end : brief → stratégie complète <30s

Phase 4 — Fine-Tuning

- ✓ Dataset 1500+ paires validé
- ✓ Modèle fine-tuné sur Hugging Face Hub
- ✓ Préférence humaine >70% vs modèle de base

Phase 5 — Interface

- ✓ Interface Next.js avec streaming fonctionnel
- ✓ Génération page vitrine HTML <60 secondes

Phase 6 — Startup

- ✓ 10 beta-utilisateurs actifs avec feedback
- ✓ Infrastructure <50€/mois
- ✓ Pitch deck prêt

7. Ressources d'Apprentissage

Toutes ces ressources sont gratuites et suffisantes pour maîtriser chaque composant.

7.1 LangChain & LangGraph

- Documentation officielle : python.langchain.com/docs
- LangGraph Academy : academy.langchain.com (cours gratuit)
- YouTube : 'LangChain Crash Course' par Greg Kamradt
- GitHub : [langchain-ai/langgraph](https://github.com/langchain-ai/langgraph) (exemples officiels)

7.2 RAG

- 'RAG from Scratch' — série YouTube de LangChain (15 vidéos)
- LlamaIndex Documentation : docs.llamaindex.ai
- Paper : 'Retrieval-Augmented Generation' (Lewis et al. 2020) — gratuit sur arXiv
- ChromaDB Docs : docs.trychroma.com

7.3 Fine-Tuning

- Unsloth GitHub : github.com/unslothai/unsloth (notebooks Colab inclus)
- Hugging Face Course : huggingface.co/learn/nlp-course (gratuit)
- 'Fine-Tuning LLMs' — cours gratuit DeepLearning.AI
- Paper : 'LoRA: Low-Rank Adaptation' — arXiv 2106.09685

7.4 FastAPI & Next.js

- FastAPI Tutorial officiel : fastapi.tiangolo.com/tutorial
- Next.js Learn : nextjs.org/learn (interactif, gratuit)
- Full Stack FastAPI + Next.js : youtube.com (nombreux tutoriels)

Conseil d'organisation

Ne pas tout apprendre avant de coder. Apprends le minimum pour chaque phase, code, bloque, cherche la solution spécifique. L'apprentissage par la pratique est 3x plus efficace pour ce type de projet.

8. Risques & Solutions

Risque identifié	Solution recommandée
Rate limits des APIs gratuites	Implémenter un système de fallback : Groq → Gemini → Ollama
Mauvaise qualité du RAG	Améliorer le chunking, ajouter un reranker, enrichir les données
Fine-tuning dépasse le GPU Colab	QLoRA 4-bit + gradient checkpointing + batch size réduit
Agent en boucle infinie	Ajouter un max_iterations et un timeout sur chaque noeud LangGraph
Coûts inattendus sur les APIs	Monitorer avec LangSmith, définir des limites strictes par utilisateur
Projet trop ambitieux	Lancer avec un MVP minimal : juste le RAG + 3 prompts optimisés
Manque de données marketing	Commencer avec 50 documents de haute qualité, puis enrichir

Conclusion & Prochaine étape

Tu as maintenant une roadmap complète, une stack technique gratuite et toutes les ressources nécessaires pour construire MarketAI de A à Z. Ce projet te permet de démontrer, en un seul livrable, la maîtrise des 4 compétences fondamentales d'un AI Engineer : RAG, Agent, Prompt Engineering et Fine-Tuning.

Mais au-delà du projet de fin de formation, tu as la possibilité de transformer ça en une véritable startup. Il y a un besoin réel, un marché sous-équipé, et une fenêtre de tir avant que les géants du logiciel ne comblerent ce vide.

Ta première action concrète — aujourd'hui

- 36. Créer le dépôt GitHub avec la structure décrite en Phase 0
- 37. Ouvrir les 5 comptes gratuits (Groq, HuggingFace, Supabase, Vercel, Railway)
- 38. Collecter les 10 premiers articles de HubSpot et les sauvegarder en .txt

Ces 3 actions prennent moins de 2 heures et lancent concrètement le projet.